Musterlösung zur Übung 3

Einstieg in Java

Programmierung 1 · WS 2026/27
Bachelor Cyber Security & Privacy · André Kless, M.Sc.

Hinweis: Die Lösungen zeigen einen möglichen Weg. Andere Programme sind ebenfalls richtig, wenn sie alle Vorgaben erfüllen. Alle vollständigen Programme verwenden Java 25 oder höher und eine kompakte `void main()`.

Grundlage: Das Übungsblatt zu Kapitel 3. Zahleneingaben werden wie dort vorausgesetzt als korrekt geschriebene `int`-Werte eingegeben. Zusätzliche Wertebereiche prüfen die Programme selbst.

01 Java einrichten und im Terminal starten | Basic

a) Nach der Installation müssen `java --version` und `javac --version` jeweils mindestens Version 25 anzeigen. Die genaue Ausgabe hängt von der installierten Version ab. Werden die Befehle nicht gefunden, ist die Einrichtung anhand der Installationsfolie zu prüfen.

b–c) Beispiel für den Arbeitsordner und die Datei `Hallo.java`:

```
mkdir java-start
cd java-start
```

```
void main() {
    IO.println("Hallo Java!");
}
```

Die Datei im Texteditor als `Hallo.java` im Arbeitsordner speichern. Dort im Terminal übersetzen und starten:

```
javac Hallo.java
java Hallo
```

Ausgabe: `Hallo Java!`. Der Compiler erzeugt `Hallo.class` mit Bytecode. Die JVM lädt diese Datei und führt den Bytecode aus. Ein Interpreter kann ihn schrittweise ausführen; ein JIT-Compiler kann zur Laufzeit häufig genutzte Teile in Maschinencode übersetzen.

d) Nach einer Änderung nur der Datei `Hallo.java` zeigt `java Hallo` noch die alte Ausgabe: Die vorhandene Datei `Hallo.class` ist unverändert. Erst `javac Hallo.java` erzeugt den aktualisierten Bytecode; der anschließende Start zeigt den neuen Text. Hier wird ausdrücklich mit `java Hallo` gestartet, nicht mit `java Hallo.java`.

02 In der IDE weiterarbeiten | Basic

a–b) Im Projekt `Prog1` ist JDK 25 oder höher ausgewählt. In `src` liegt das Java Compact File `Aufgabe02.java` ohne Package. Eine mögliche Lösung lautet:

```
void main() {
    // Drei Zeilen mit einer kurzen Vorstellung ausgeben.
    IO.println("Alex Beispiel");
    IO.println("Cyber Security & Privacy");
    IO.println("Guten Start in Programmierung 1!");
}
```

c) Ein fehlendes Semikolon verletzt die Syntax; eine typische Meldung lautet `';' expected`. Nach dem Ergänzen lässt sich das Programm wieder übersetzen. Ein geänderter Text innerhalb korrekt gesetzter Anführungszeichen bleibt syntaktisch gültig. Er verändert die Ausgabe und damit die Wirkung des Programms.

Basic: Deklaration und Ein- und Ausgabe

03 Einen Einkauf berechnen | Basic

a–c) Vollständige Lösung in `Aufgabe03.java`. Der Gesamtpreis wird hier erst deklariert und anschließend durch eine Zuweisung berechnet:

```
void main() {
    String name = IO.readln("Name: ");
    int stueckzahl = Integer.parseInt(IO.readln("Stückzahl: "));
    final double STUECKPREIS = 2.5;
    double gesamtpreis;
    gesamtpreis = stueckzahl * STUECKPREIS;
    boolean grosseBestellung = stueckzahl >= 10;

    IO.println("Name: " + name);
    IO.println("Stückzahl: " + stueckzahl);
    IO.println("Gesamtpreis: " + gesamtpreis + " Euro");
    IO.println("Mindestens zehn Hefte: " + grosseBestellung);
}
```

`IO.readln()` liefert Text. `Integer.parseInt()` wandelt den Text der Stückzahl in eine ganze Zahl um. Durch den `double`-Stückpreis liefert die Multiplikation einen `double`-Wert. Der Vergleich `stueckzahl >= 10` liefert einen `boolean`-Wert.

c) Gleichwertige Deklaration mit Initialisierung, die die beiden Zeilen für `gesamtpreis` ersetzt:

```
double gesamtpreis = stueckzahl * STUECKPREIS;
```

Im Syntaxdiagramm zur Deklaration wird nach Typ und Bezeichner entweder der Weg ohne Initialisierung oder der Weg über = und <Ausdruck> gewählt. Beide Deklarationen enden mit `;`. Bei der ersten Variante ist die folgende Zuweisung eine zusätzliche Einzelanweisung. `final` verhindert eine spätere Neuzuweisung an `STUECKPREIS`.

d) Erwartete Ergebnisse; der eingegebene Name wird unverändert ausgegeben:

Stückzahl	Gesamtpreis in Euro	Mindestens zehn Hefte
1	2.5	false
3	7.5	false
9	22.5	false
10	25.0	true
100	250.0	true

Die Stückzahlen liegen wie vorausgesetzt im Bereich 1 bis 100. Insbesondere prüfen 9 und 10 den Wechsel des Wahrheitswerts an der Grenze.

Basic: Selektion und Schaltjahre

04 Schaltjahre bestimmen | Basic

a) Lösung mit Fallunterscheidung in `Aufgabe04If.java`. Die Eingabe liegt wie vorausgesetzt zwischen 1 und 9999. Vor 1582 gilt im vereinfachten Modell die julianische Regel, ab 1582 die gregorianische.

```
void main() {
    int jahr = Integer.parseInt(IO.readln("Jahr (1 bis 9999): "));
    boolean schaltjahr;
    if (jahr < 1582) {
        if (jahr % 4 == 0) {
            schaltjahr = true;
        } else {
            schaltjahr = false;
        }
    } else {
        if (jahr % 400 == 0) {
            schaltjahr = true;
        } else if (jahr % 100 == 0) {
            schaltjahr = false;
        } else if (jahr % 4 == 0) {
            schaltjahr = true;
        } else {
            schaltjahr = false;
        }
    }
    IO.println("Schaltjahr: " + schaltjahr);
}
```

`jahr % 4 == 0` bedeutet, dass bei der Division durch 4 kein Rest bleibt. Entsprechend prüfen die Vergleiche mit 100 und 400 die Teilbarkeit durch diese Zahlen. Im gregorianischen Zweig wird zuerst die Ausnahme für durch 400 teilbare Jahre geprüft, danach die Jahrhundertregel und schließlich die Vierjahresregel.

Für jeden möglichen Weg erhält `schaltjahr` einen Wert. Die Ausgabe steht nach der Selektion; deshalb ist die Variable vor der Selektion deklariert.

Basic: Schaltjahre als logischer Ausdruck

04 Schaltjahre bestimmen — Fortsetzung | Basic

b) Vollständige zweite Variante in `Aufgabe04Logisch.java`:

```
void main() {
    int jahr = Integer.parseInt(IO.readln("Jahr (1 bis 9999): "));
    boolean schaltjahr = jahr % 4 == 0
            && (jahr < 1582 || jahr % 100 != 0 || jahr % 400 == 0);
    IO.println("Schaltjahr: " + schaltjahr);
}
```

Ein Jahr muss in beiden Kalendern durch 4 teilbar sein. Zusätzlich muss mindestens eine der drei Aussagen in der Klammer zutreffen: Das Jahr liegt vor 1582; es ist nicht durch 100 teilbar; oder es ist durch 400 teilbar. Für julianische Jahre genügt bereits die erste Aussage. Bei gregorianischen Jahren bleiben die beiden Jahrhundertbedingungen wirksam.

Erwartete Ergebnisse beider Varianten:

Jahr	Regel im Modell	Schaltjahr	Begründung
1500	julianisch	true	Durch 4 teilbar; die Jahrhundertregel gilt hier nicht.
1582	gregorianisch	false	Nicht durch 4 teilbar.
1600	gregorianisch	true	Durch 400 teilbar.
1700	gregorianisch	false	Durch 100, aber nicht durch 400 teilbar.
1900	gregorianisch	false	Durch 100, aber nicht durch 400 teilbar.
2000	gregorianisch	true	Durch 400 teilbar.
2024	gregorianisch	true	Durch 4, aber nicht durch 100 teilbar.
2025	gregorianisch	false	Nicht durch 4 teilbar.

Besonders aussagekräftig sind 1500 für die julianische Regel sowie 1900 und 2000 für den Unterschied zwischen Jahrhundertregel und 400-Jahre-Ausnahme. 1582 gehört ausdrücklich bereits zum gregorianischen Teil unseres Modells.

Basic: Den Zune-Fehler im Debugger nachvollziehen

04 Schaltjahre bestimmen — Fehleranalyse | Basic

c) Bei 2008 ist die Schaltjahrprüfung korrekt: `schaltjahr` wird `true`. Der Fehler liegt im Schleifenablauf der Datumsumrechnung, nicht in dieser Prüfung.

Startwert tage	Beobachtung im fehlerhaften Programm	Ergebnis
365	Die Bedingung tage > 365 ist sofort falsch.	Jahr 2008, Tag 365
366	tage > 365 ist wahr, schaltjahr ist true, tage > 366 ist falsch. Keine Variable wird verändert.	Kein Ende und keine Ausgabe
367	Es werden 366 Tage abgezogen; jahr wird 2009, tage wird 1.	Jahr 2009, Tag 1

Bei Startwert 366 zeigt jeder weitere Halt am Breakpoint erneut `jahr = 2008` und `tage = 366`. Nach der Berechnung ist `schaltjahr` erneut `true`. Der innere `if`-Zweig wird nicht betreten; anschließend beginnt derselbe Durchlauf wieder. Nach zwei beobachteten Wiederholungen kann der Debugger mit „Stop“ beendet werden.

Korrektur in `Aufgabe04ZuneKorrigiert.java`: Ein ergänztes `else` mit `break` beendet die Schleife, wenn der letzte Tag des Schaltjahres bereits erreicht ist.

```
void main() {
    int jahr = 2008;
    int tage = 366;
    while (tage > 365) {
        boolean schaltjahr = jahr % 4 == 0
                && (jahr % 100 != 0 || jahr % 400 == 0);
        if (schaltjahr) {
            if (tage > 366) {
                tage -= 366;
                jahr++;
            } else {
                break;
            }
        } else {
            tage -= 365;
            jahr++;
        }
    }
    IO.println("Jahr: " + jahr + ", Tag im Jahr: " + tage);
}
```

Nach der Korrektur lauten die Ergebnisse: 365 → Jahr 2008, Tag 365; 366 → Jahr 2008, Tag 366; 367 → Jahr 2009, Tag 1. Die Tagesnummer 366 bezeichnet den 31. Dezember 2008.

Nur > durch >= zu ersetzen würde zwar die Endlosschleife verhindern, aber bei Startwert 366 das falsche Ergebnis „Jahr 2009, Tag 0“ erzeugen. Der Tag 366 gehört noch zum Schaltjahr; erst Tag 367 gehört zum Folgejahr.

Zeitgenössischer Pressebericht: The Guardian — Bericht vom 1. Januar 2009

Veröffentlichter C-Ablauf und Fehleranalyse: James Grenning — Zune Bug: Test Driven Bug Fix (2009)

Combined: Zwei Formen der Mehrfachselektion

05 Ein Menü mit zwei Schreibweisen | Combined

a) Doppelpunktform in `Aufgabe05Doppelpunkt.java`:

```
void main() {
    int auswahl = Integer.parseInt(IO.readln("Auswahl (1 bis 3): "));
    switch (auswahl) {
        case 1:
            IO.println("Öffnungszeiten");
            break;
        case 2:
            IO.println("Kontakt");
            break;
        case 3:
            IO.println("Anfahrt");
            break;
        default:
            IO.println("Ungültige Auswahl");
    }
}
```

b) Fehlt nur das erste `break`, gibt Auswahl 1 nacheinander `Öffnungszeiten` und `Kontakt` aus. Nach dem passenden Fall läuft die Ausführung in den nächsten Fall weiter, ohne dessen Fallwert erneut zu prüfen. Das `break` nach Fall 2 beendet dann die Mehrfachselektion. Dieses Weiterlaufen heißt Fall-through.

c) Pfeilform in `Aufgabe05Pfeil.java`:

```
void main() {
    int auswahl = Integer.parseInt(IO.readln("Auswahl (1 bis 3): "));
    switch (auswahl) {
        case 1 -> IO.println("Öffnungszeiten");
        case 2 -> IO.println("Kontakt");
        case 3 -> IO.println("Anfahrt");
        default -> IO.println("Ungültige Auswahl");
    }
}
```

Auswahl	Ausgabe beider korrekter Varianten
1	Öffnungszeiten
2	Kontakt
3	Anfahrt
9	Ungültige Auswahl

Für dieses Menü ist die Pfeilform besonders übersichtlich: Jeder Fall steht für eine unabhängige Aktion. Ein versehentlich fehlendes `break` kann kein Weiterlaufen auslösen. Die Doppelpunktform ist ebenfalls korrekt; ihr gezieltes Weiterlaufen wird hier nicht benötigt.

Combined: Ein Ratespiel mit begrenzten Versuchen

06 Ein Zahlenratespiel entwickeln | Combined

a–b) Vollständige Lösung in `Aufgabe06.java`. Die erste Schleife prüft die obere Grenze; die zweite verarbeitet die Tipps:

```
void main() {
    int n;
    do {
        n = Integer.parseInt(IO.readln("Obere Grenze n: "));
        if (n < 1) {
            IO.println("Bitte mindestens 1 eingeben.");
        }
    } while (n < 1);

    int geheimzahl = (int) (Math.random() * n) + 1;
    int versuche = 0;
    boolean erraten = false;
    while (versuche < 10 && !erraten) {
        int tipp = Integer.parseInt(IO.readln("Dein Tipp: "));
        versuche++;
        if (tipp < geheimzahl) {
            IO.println("Deine Zahl ist zu klein.");
        } else if (tipp > geheimzahl) {
            IO.println("Deine Zahl ist zu groß.");
        } else {
            erraten = true;
            IO.println("Richtig! Benötigte Versuche: " + versuche);
        }
    }
    if (!erraten) {
        IO.println("Keine Versuche mehr. Gesucht war: " + geheimzahl);
    }
}
```

a) Die Eingabeschleife wird mindestens einmal durchlaufen. Erst wenn `n >= 1` gilt, wird die geheime Zahl erzeugt. Sie bleibt für das gesamte Spiel unverändert. Die Eingaben für `n` zählen nicht als Rateversuche.

b) `versuche` zählt die bereits eingelesenen Tipps. `erraten` merkt sich, ob ein Treffer vorliegt. Die Rateschleife läuft nur weiter, wenn noch Versuche übrig sind und die Zahl noch nicht erraten wurde.

Combined: Spielende und gezielte Tests

06 Ein Zahlenratespiel entwickeln — Fortsetzung | Combined

b) Der Zähler wird nach jeder Eingabe erhöht. Beim ersten Tipp hat `versuche` deshalb den Wert 1, beim zehnten den Wert 10. Auch der zehnte Tipp wird noch geprüft. Bei einem Treffer wird `erraten` auf `true` gesetzt; anschließend endet die Schleife, und die abschließende Fehlermeldung entfällt.

Bei zehn falschen Tipps bleibt `erraten` gleich `false`. Die Bedingung `versuche < 10` ist nun falsch. Die Schleife endet, und die gesuchte Zahl wird ausgegeben. Nach einem falschen zehnten Tipp erscheint davor noch die Rückmeldung „zu klein“ oder „zu groß“. Eine elfte Eingabe wird nicht mehr angefordert.

c) Für reproduzierbare Tests mit `n = 10` wird genau die Zeile zur Zufallszahl durch Folgendes ersetzt:

```
int geheimzahl = 7;
```

Jede Tabellenzeile beschreibt einen neuen Programmstart. Die aufgeführten Eingaben sind nur die Tipps; vorher wird jeweils `10` für `n` eingegeben.

Test	Tipps	Erwartetes Verhalten
Sofortiger Treffer	`7`	Erfolg nach 1 Versuch; kein weiterer Tipp.
Beide Rückmeldungen	`5, 9, 7`	Zu klein, zu groß, Erfolg nach 3 Versuchen.
Treffer beim letzten Versuch	`neunmal 1, dann 7`	Neunmal zu klein; Erfolg nach 10 Versuchen.
Alle Versuche verbraucht	`zehnmal 1`	Zehnmal zu klein; danach Auflösung 7. Kein elfter Tipp.

Die Eingabefolge `-1, 0, 10` für `n` erzeugt zweimal den Hinweis und beginnt erst danach das Spiel. Die Anzahl der Rateversuche steht zu diesem Zeitpunkt noch auf 0.

Für `n = 1` wird die ursprüngliche Zufallszahl-Zeile verwendet. Sie erzeugt immer 1. Der Tipp `1` muss deshalb sofort zum Erfolg führen. Nach allen Tests wird die Zufallszahl-Zeile dauerhaft wiederhergestellt.

Die Multiplikation von `Math.random()` mit `n` ergibt einen nichtnegativen Wert unterhalb von `n`. `(int)` entfernt hier den Nachkommateil, sodass 0 bis `n - 1` entstehen; die Addition von 1 ergibt den gewünschten Bereich. Die Klammern sind wichtig: Würde zuerst nur `Math.random()` in `int` umgewandelt, wäre das Ergebnis immer 0.

Combined: Debugging und Schleifensteuerung

07 Einen Schleifenfehler im Debugger finden | Combined

a) Gewünscht ist `1 + 2 + 3 + 4 + 5 = 15`. Tatsächlich wird nur bis 4 addiert, also 10 ausgegeben. Das Programm ist syntaktisch gültig; die Bedingung beschreibt aber den falschen Wertebereich. Es ist ein Fehler in der Programmlogik.

b) Am Breakpoint vor `summe += i;` wurde die Addition des aktuellen Durchlaufs noch nicht ausgeführt:

i	Summe vor Addition	Summe nach Addition
1	0	1
2	1	3
3	3	6
4	6	10

Im Debugger jeweils die angehaltene Zeile mit „Step Over“ ausführen und danach den geänderten Wert von `summe` ablesen. Für `i = 5` wird der Breakpoint nicht mehr erreicht, weil `i < 5` dann falsch ist.

c) Korrigiertes vollständiges Programm `Aufgabe07.java`:

```
void main() {
    int summe = 0;
    for (int i = 1; i <= 5; i++) {
        summe += i;
    }
    IO.println(summe);
}
```

Nun wird für `i = 5` die Summe von 10 auf 15 erhöht. Die Ausgabe lautet `15`.

d) Den Rumpf der korrigierten Schleife durch Folgendes ersetzen:

```
if (i == 3) {
    continue;
}
summe += i;
```

Mit `continue` entfällt nur die Addition von 3: `1 + 2 + 4 + 5 = 12`. Anschließend führt die `for`-Schleife ihre Aktualisierung `i++` aus und prüft erneut die Bedingung. Ersetzt man ausschließlich `continue` durch `break`, endet die Schleife bei 3 vollständig. Addiert wurden dann nur 1 und 2; die Ausgabe lautet `3`.

Combined: Geschachtelte Schleifen

08 Punkte eines Gitters ausgeben | Combined

a–b) Vollständige Lösung `Aufgabe08.java` mit Zähler:

```
void main() {
    int anzahl = 0;
    for (int y = 0; y < 3; y++) {
        for (int x = 0; x < 4; x++) {
            IO.println("(" + x + "," + y + ")");
            anzahl++;
        }
    }
    IO.println("Anzahl: " + anzahl);
}
```

a) Erwartete Ausgabe:

```
(0,0)
(1,0)
(2,0)
(3,0)
(0,1)
(1,1)
(2,1)
(3,1)
(0,2)
(1,2)
(2,2)
(3,2)
Anzahl: 12
```

b) `anzahl` wird vor beiden Schleifen im Rumpf von `main` deklariert. Dadurch ist die Variable in beiden Schleifen und bei der abschließenden Ausgabe sichtbar. Für jeden der drei Werte von `y` nimmt `x` vier Werte an: insgesamt zwölf Punkte.

c) Der Rumpf der inneren Schleife wird ersetzt durch:

```
if (x == y) {
    continue;
}
IO.println("(" + x + "," + y + ")");
anzahl++;
```

Es entfallen `(0,0)`, `(1,1)` und `(2,2)`. Alle anderen Punkte erscheinen in derselben Reihenfolge wie zuvor. Der Zähler liefert `Anzahl: 9`. Weil die Erhöhung hinter `continue` steht, werden übersprungene Punkte nicht mitgezählt. `continue` betrifft hier die innere Schleife.

Challenge: Den ggT auf drei Zahlen erweitern

09 Den ggT für drei Zahlen berechnen | Challenge

a–b) Das vollständige Programm `Aufgabe09.java` steht auf dieser und der folgenden Seite. Zunächst werden die drei Eingaben unabhängig geprüft; ungültige Werte führen nur zur Wiederholung der jeweiligen Eingabe.

```
void main() {
    int a;
    do {
        a = Integer.parseInt(IO.readln("a (1 bis 1000): "));
    } while (a < 1 || a > 1000);
    int b;
    do {
        b = Integer.parseInt(IO.readln("b (1 bis 1000): "));
    } while (b < 1 || b > 1000);
    int c;
    do {
        c = Integer.parseInt(IO.readln("c (1 bis 1000): "));
    } while (c < 1 || c > 1000);

    int x = a;
    int y = b;
    int subtraktionen1 = 0;
    while (x != y) {
        if (x > y) {
            x -= y;
        } else {
            y -= x;
        }
        subtraktionen1++;
    }
    int ggTab = x;
```

Die ursprünglichen Werte `a`, `b` und `c` werden nach der Eingabe nicht verändert. Die erste Berechnung arbeitet mit `x` und `y`. Nach ihrem Ende ist `x == y`; dieser gemeinsame Wert ist der ggT von `a` und `b`. Jede Ausführung des Schleifenrumpfs führt genau eine Subtraktion aus.

Fortsetzung auf der nächsten Seite: Sie gehört unmittelbar hinter die letzte Zeile, weiterhin in dieselbe `main`.

Challenge: Zweite Berechnungsphase und Tests

09 Den ggT für drei Zahlen berechnen — Fortsetzung | Challenge

b–d) Fortsetzung von `Aufgabe09.java`, einschließlich Ergebnis und Zählern:

```
    y = c;
    int subtraktionen2 = 0;
    while (x != y) {
        if (x > y) {
            x -= y;
        } else {
            y -= x;
        }
        subtraktionen2++;
    }
    IO.println("ggT(" + a + "," + b + "," + c + ") = " + x);
    IO.println("ggT von a und b: " + ggTab);
    IO.println("Subtraktionen in Phase 1: " + subtraktionen1);
    IO.println("Subtraktionen in Phase 2: " + subtraktionen2);
}
```

Die zweite Phase beginnt mit dem bereits in `x` gespeicherten ggT von `a` und `b` sowie `c` in `y`. Am Ende steht in `x` der gemeinsame ggT aller drei Zahlen.

c–d) Erwartete Ergebnisse und Anzahl der Subtraktionen:

Eingaben (a,b,c)	ggT(a,b)	ggT(a,b,c)	Phase 1	Phase 2
(18,12,9)	6	3	2	2
(7,7,7)	7	7	0	0
(1,9,6)	1	1	8	5
(21,14,35)	7	7	2	4
(12,18,9)	6	3	2	2

d) Beim Vertauschen von 18 und 12 wechselt lediglich, welche Hilfsvariable zuerst verkleinert wird. Beide Phasen benötigen weiterhin jeweils zwei Subtraktionen. Gleiche Zahlen wie 7 und 7 benötigen keine Subtraktion.

Beim Zweizahlenfall `(0,5)` würde immer wieder `5 - 0` berechnet. Die Werte ändern sich nicht, und die Bedingung `x != y` bleibt wahr. Die Schleife würde nicht enden. Deshalb lässt die Eingabeprüfung nur positive Werte zu.

Beispiel für die Eingabeprüfung: Bei der Folge 0, 1001, 18 wird dreimal nach `a` gefragt; erst 18 wird übernommen. Danach werden `b` und `c` eingelesen. Ein ungültiges `b` setzt die bereits akzeptierte Eingabe von `a` nicht zurück.

Challenge: Alle passenden Packungen suchen

10 Passende Packungen finden | Challenge

a) Sei `vierer` die Anzahl der Viererpackungen und `sechser` die Anzahl der Sechserpackungen. Eine Kombination passt genau dann, wenn `4 * vierer + 6 * sechser == anzahl`. Die Obergrenzen `anzahl / 4` und `anzahl / 6` genügen: Mehr Packungen würden allein schon zu viele Plätze bereitstellen. Die Division von positiven `int`-Werten liefert hier den ganzzahligen Anteil.

a–b) Vollständige Lösung `Aufgabe10Alle.java`:

```
void main() {
    int anzahl;
    do {
        anzahl = Integer.parseInt(IO.readln("Artikel (1 bis 100): "));
    } while (anzahl < 1 || anzahl > 100);

    int moeglichkeiten = 0;
    for (int sechser = 0; sechser <= anzahl / 6; sechser++) {
        for (int vierer = 0; vierer <= anzahl / 4; vierer++) {
            if (4 * vierer + 6 * sechser == anzahl) {
                IO.println("Vierer: " + vierer + ", Sechser: " + sechser);
                moeglichkeiten++;
            }
        }
    }
    if (moeglichkeiten == 0) {
        IO.println("Keine passende Kombination");
    } else {
        IO.println("Möglichkeiten: " + moeglichkeiten);
    }
}
```

b) Jedes Paar von Packungsanzahlen wird genau einmal geprüft. Die Reihenfolge einzelner Packungen wird nicht unterschieden.

Artikel	Passende Paare (Vierer,Sechser)	Anzahl
1	keine passende Kombination	0
4	(1,0)	1
6	(0,1)	1
8	(2,0)	1
12	(3,0), (0,2)	2
24	(6,0), (3,2), (0,4)	3

Bei 1 erscheint nur die Meldung, bei den übrigen Tests erscheinen die passenden Paare und anschließend die Anzahl der Möglichkeiten.

Challenge: Eine geschachtelte Suche beenden

10 Passende Packungen finden — Fortsetzung | Challenge

c) Vollständige zweite Variante in `Aufgabe10Erster.java`. Die Sprungmarke `suche` steht unmittelbar vor der äußeren Schleife:

```
void main() {
    int anzahl;
    do {
        anzahl = Integer.parseInt(IO.readln("Artikel (1 bis 100): "));
    } while (anzahl < 1 || anzahl > 100);

    boolean gefunden = false;
    suche:
    for (int sechser = 0; sechser <= anzahl / 6; sechser++) {
        for (int vierer = 0; vierer <= anzahl / 4; vierer++) {
            if (4 * vierer + 6 * sechser == anzahl) {
                IO.println("Vierer: " + vierer + ", Sechser: " + sechser);
                gefunden = true;
                break suche;
            }
        }
    }
    if (!gefunden) {
        IO.println("Keine passende Kombination");
    }
}
```

Bei 24 wird zuerst die Kombination aus sechs Viererpackungen und null Sechserpackungen gefunden. Solange `sechser` noch 0 ist, prüft die innere Schleife `vierer` aufsteigend von 0 bis 6. Der erste Treffer beendet mit `break suche;` die gesamte äußere Schleife einschließlich der inneren Schleife.

d) Ein einfaches `break;` würde nur die innere Schleife beenden. Die äußere Schleife liefe mit der nächsten Anzahl an Sechserpackungen weiter und könnte weitere Treffer ausgeben. Das erfüllt die Forderung, nach dem ersten Treffer beide Schleifen zu beenden, nicht.

Die Variable `gefunden` wird vor den Schleifen deklariert. Sie merkt sich, ob ein Treffer vorliegt, und ist auch nach den Schleifen sichtbar. Bei einer unmöglichen Artikelanzahl bleibt sie `false`; dann erscheint die Fehlermeldung.

Artikel	Ausgabe der zweiten Variante
1	`Keine passende Kombination`
4	`Vierer: 1, Sechser: 0`
6	`Vierer: 0, Sechser: 1`
8	`Vierer: 2, Sechser: 0`
12	`Vierer: 3, Sechser: 0`
24	`Vierer: 6, Sechser: 0`